TU Delft

Bloom Filter-Based MPSI

Weekly Progress Meeting 9 April 2026

Andra Alăzăroaie

Supervisor: Lilika Markatou

Daily Supervisor: Tjitske Koster

09-04-2026

Bibliography

[1] Jelle Vos, Jorrit van Assen, Tjitske Koster, Evangelia Anna Markatou, and Zekeriya Erkin. On the insecurity of bloom filter-based private set intersections. Cryptology ePrint Archive, 2024.

[2] Ou Ruan, Changwang Yan, Jing Zhou, and Chaohao Ai. A practical multiparty private set intersection protocol based on bloom filters for unbalanced scenarios. Applied Sciences, 13(24):13215, 2023.

[3] Florian Kerschbaum. Outsourced private set intersection using homomorphic encryption. In Proceedings of the 7th ACM Symposium on Information, Computer and Communications Security, pages 85–86, 2012.

[4] Falzon, Francesca, and Evangelia Anna Markatou. "Re-visiting authorized private set intersection: A new privacy-preserving variant and two protocols." *Proceedings on Privacy Enhancing Technologies* (2025).

[5] Koster, Tjitske Ollie, Francesca Falzon, and Evangelia Anna Markatou. “Bandwidth Efficient Partial Authorized PSI.” *Cryptology ePrint Archive* (2025).

[6] Lindell, Yehuda. "How to simulate it–a tutorial on the simulation proof technique." *Tutorials on the Foundations of Cryptography: Dedicated to Oded Goldreich* (2017): 277-346.

Progress

- Implement PAPSI2
- Brainstorm on APSI1
- Continue working on the proof for APSI2

Progress

APSI 2 – Active Judge

n_t = server's set size

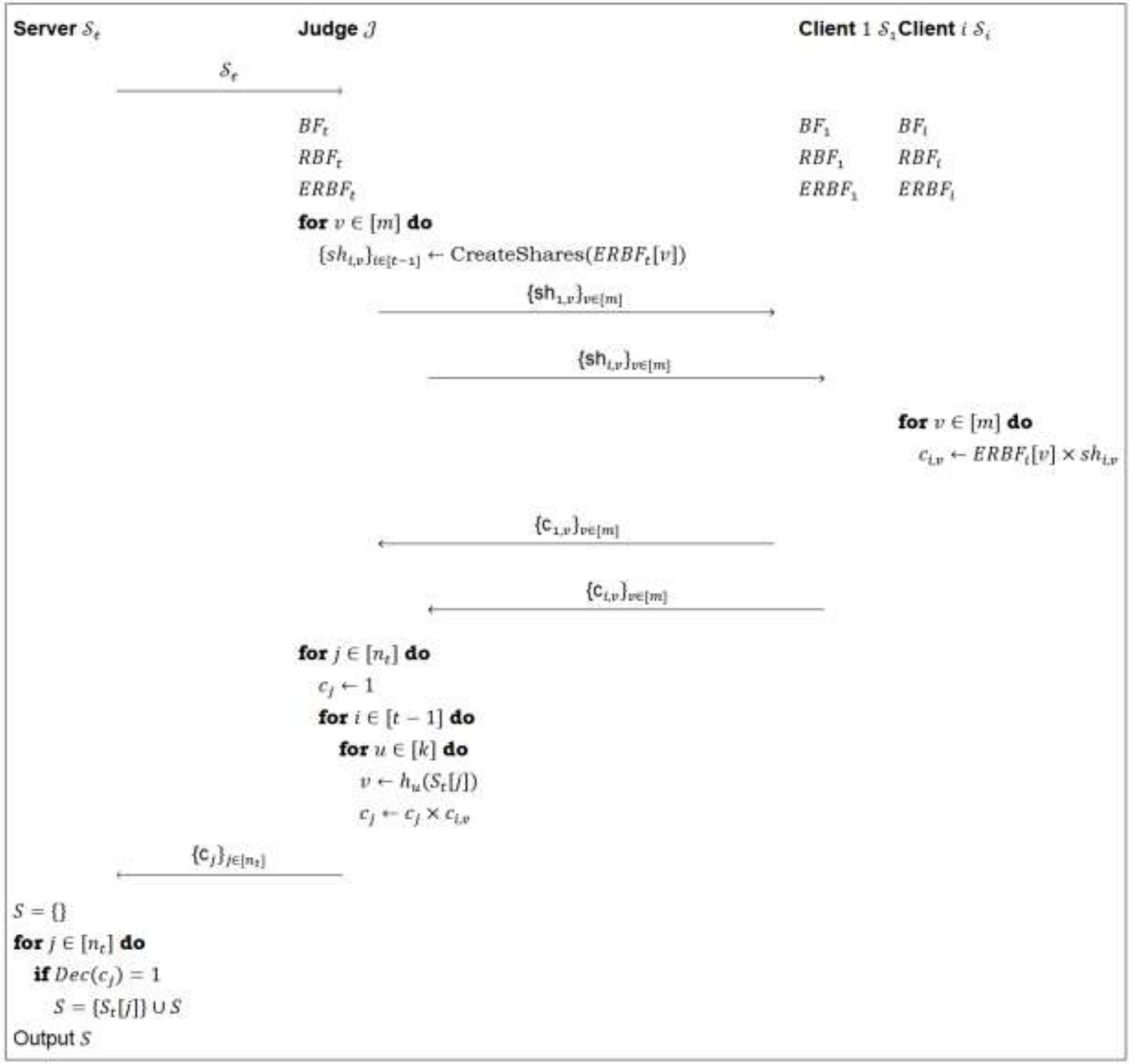
m = bin count (BF size)

k = number of hash functions

t = number of parties (including the server)

$ERBF_i$ = the encrypted randomized Bloom filter of party i

Only the Server has the decryption key

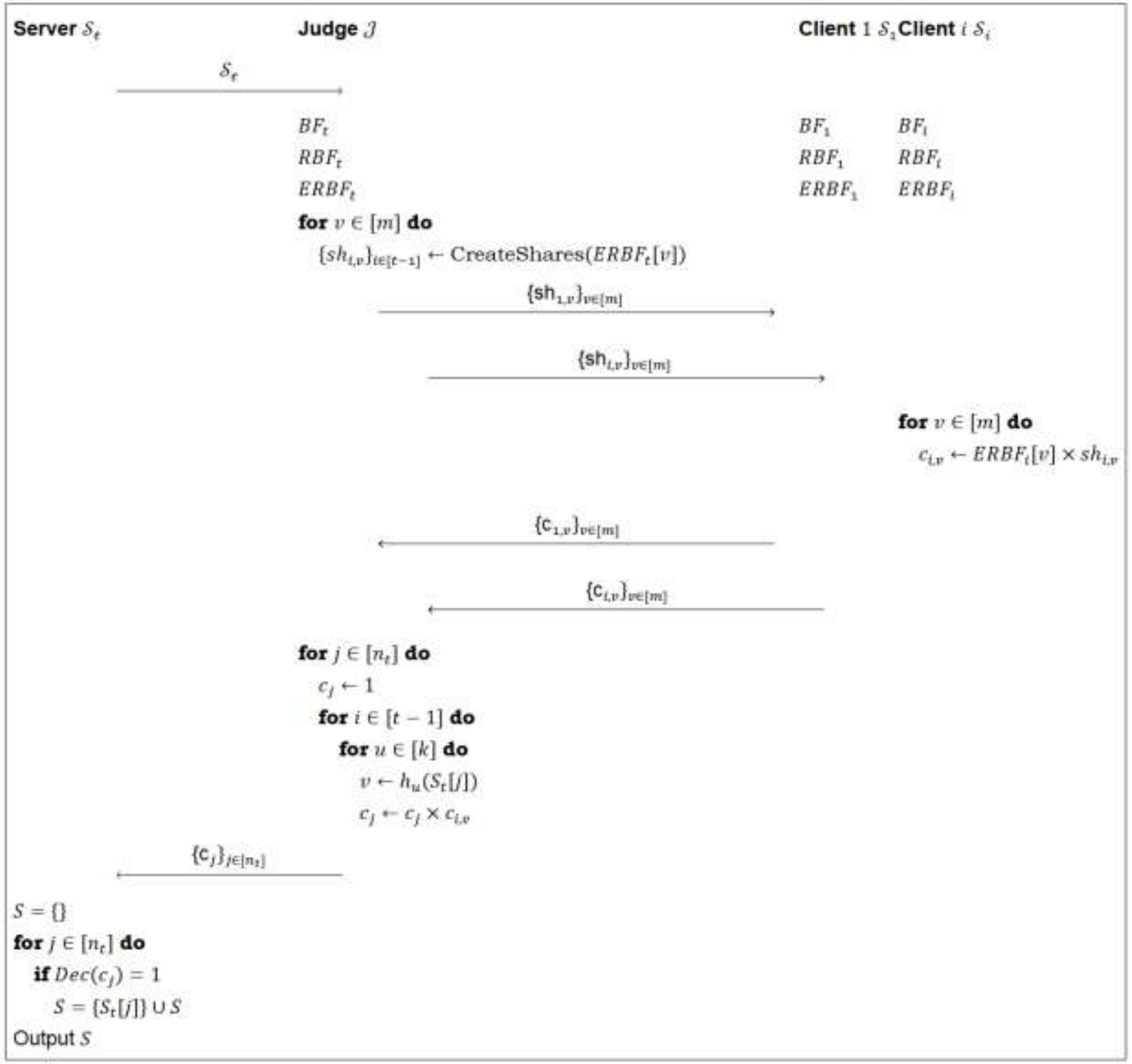


Progress

APSI 2 – Active Judge

Threat Model:

- Malicious Server
- Semi-honest Judge
- Semi-honest Clients



Progress

APSI 2 – Active Judge

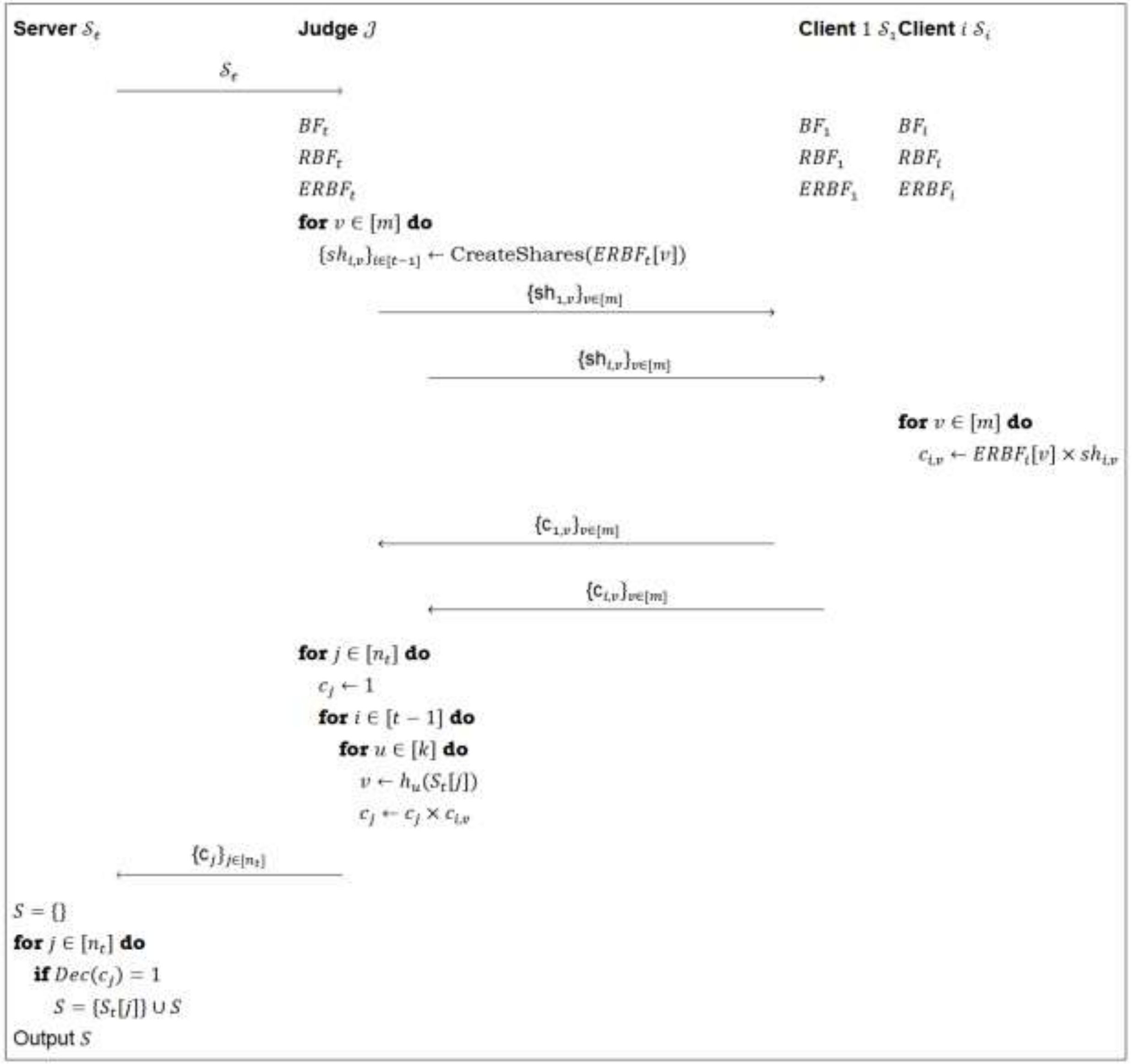
In the proof against a semi-honest judge:

- After obtaining $\{c_{i,v}\}_{i\in[t-1],v\in m}$, the judge, knowing the shares of the server's $ERBF_t$, could reverse the homomorphic multiplication to obtain $ERBF_i$:

$$\forall i \in [t-1], \forall v \in [m]:$$

$$ERBF_i[v] = c_{\{i,v\}} \times \left(sh_{\{i,v\}}\right)^{-1}$$

- The judge doesn't have the decryption key, so this is not a security issue
- However, it means that the shares are not necessary, the clients can send their encrypted BFs directly



Progress

APSI 2 – OPTIMIZED Active Judge

- The role of the judge here is only to select the appropriate bins from the encrypted BFs
- Judge does not have the decryption key, so it can infer no information about the sets of the clients

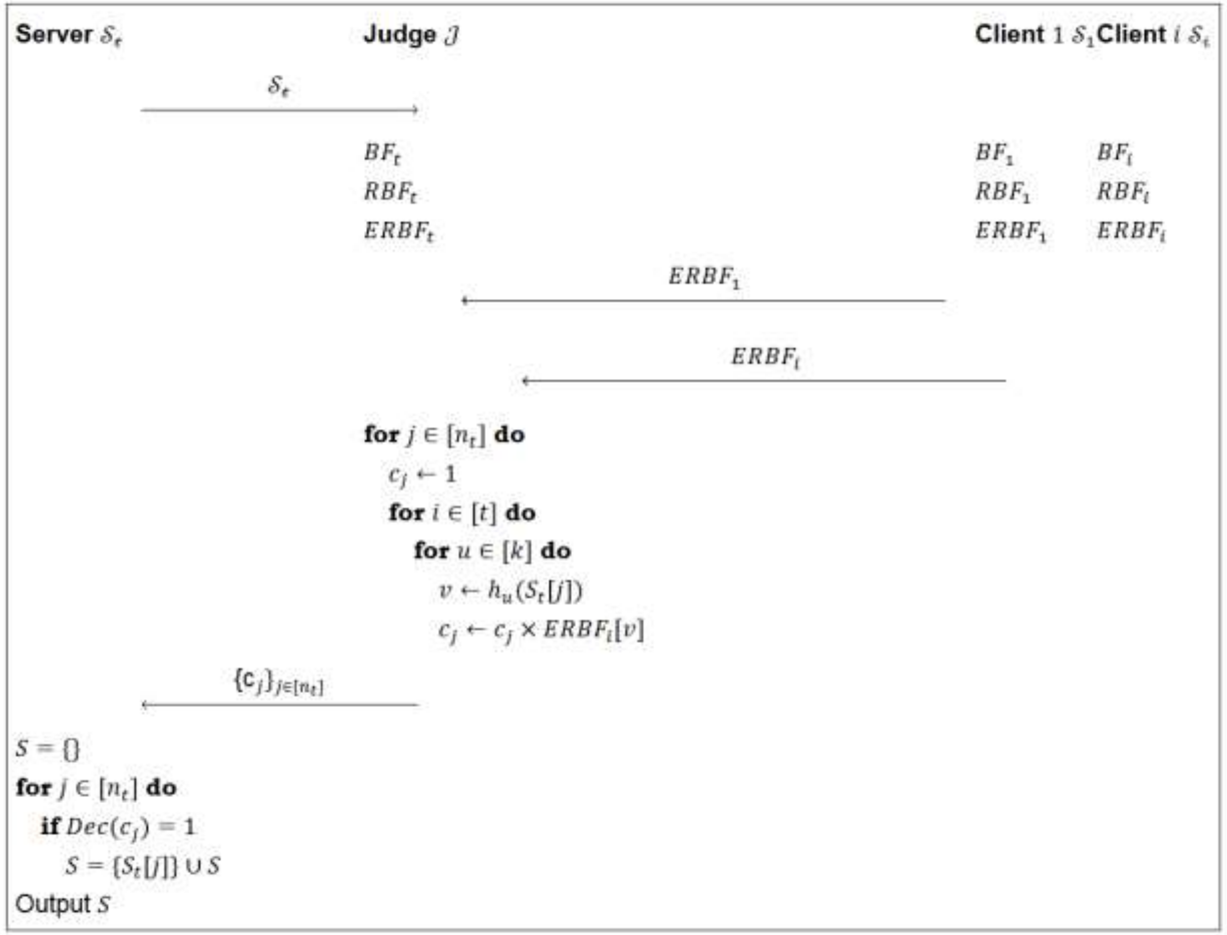


Figure 5.3: Authorized PSI with public parameters: prime number p, generator g, public key $pk = g^{sk}$ where sk is the server's secret key. BF_i represents the Bloom filter of party i, RBF_i represents their randomized Bloom filter and $ERBF_i$ their encrypted randomized Bloom filter.

Progress

APSI 2 – OPTIMIZED Active Judge

Theorem 5.1.5. *The APSI protocol is unconditionally secure against up to $t-1$ colluding semi-honest clients.*

Proof. We construct a simulator Sim_C that takes as input the clients' sets $\{S_i\}_{i=1}^{t-1}$, the server's set cardinality $|S_t|$ and the public parameters (p, g, pk). The simulator must generate the joint view of all $t-1$ clients.

In the real protocol execution, the clients receive no messages from the server or the judge. The participation of each client i is limited to computing their encrypted randomized Bloom filter ($ERBF_i$) using the server's public key and sending it to the judge.

Since the clients' joint view consists only of their own inputs, the server's set cardinality, their encryption randomness and the public parameters, simulation of this view is trivial. Because no information is ever sent to the clients, they learn nothing about the server's set or the intersection result, even under full collusion. Therefore, the simulated view is identical to the real-world view. □

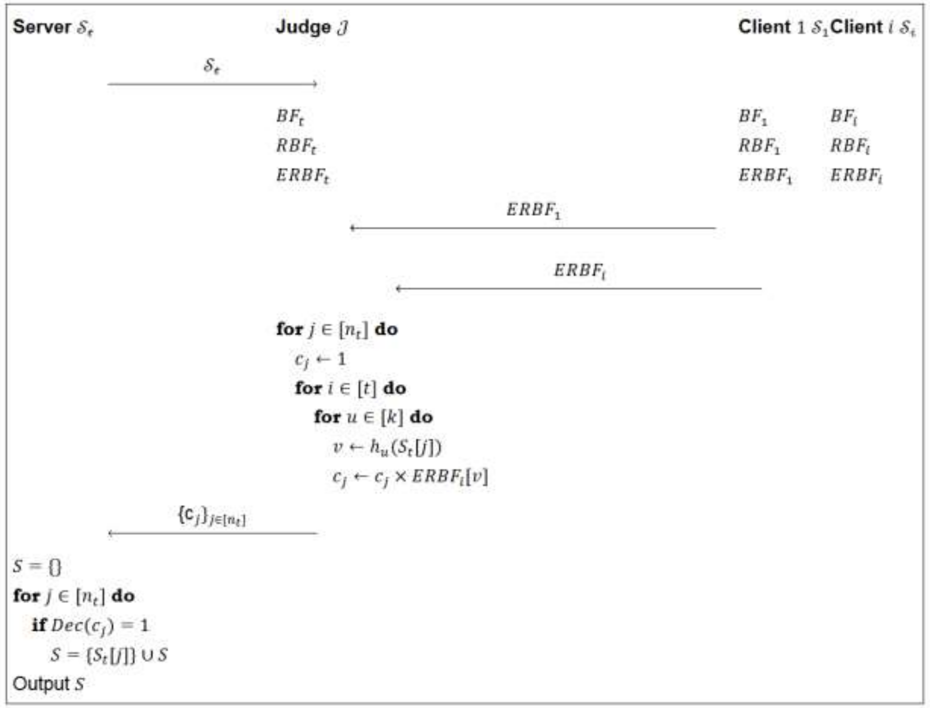


Figure 5.3: Authorized PSI with public parameters: prime number p, generator g, public key $pk = g^{sk}$ where sk is the server's secret key. BF_i represents the Bloom filter of party i, RBF_i represents their randomized Bloom filter and $ERBF_i$ their encrypted randomized Bloom filter.

Progress

APSI 2 – OPTIMIZED Active Judge

Theorem 5.1.5. *The APSI protocol is secure against a semi-honest judge $\mathcal{J}$ assuming the IND-CPA security of the encryption scheme.*

Proof. We construct a simulator $Sim_{\mathcal{J}}$ that takes as input $\{|S_i|\}_{i\in[t]}$ and the public parameters p, g, pk. The Simulator ($Sim_{\mathcal{J}}$) proceeds as follows:

1. Receive S_t in plaintext from the server.
2. Honestly compute the Bloom Filter BF_t, Randomized Bloom Filter RBF_t, Encrypted Randomized Bloom Filter $ERBF_t$ and the secret shares $\{sh_{i,v}\}_{i\in[t-1],v\in[m]}$ just as the real Judge would.
3. For each client $i \in [t-1]$:
   (a) Define a set $\hat{S}_i$ to simulate S_i.
   (b) Compute the Bloom filter $B\hat{F}_i$ corresponding to $\hat{S}_i$ honestly by inserting each element into the m bins using the k hash functions.
   (c) Define the randomized Bloom filter $R\hat{B}F_i$ by sampling random group elements for each bin $v \in [m]$ where $B\hat{F}_i[v] = 0$, and copying over the 1s for bins where $B\hat{F}_i[v] = 1$.
   (d) Define the encrypted randomized Bloom filter $ER\hat{B}F_i$ by encrypting each bin $v \in [m]$ of $R\hat{B}F_i$ using the server's public key pk.
   (e) Send $ER\hat{B}F_i$ to $\mathcal{J}$.

We show that the joint view generated by the simulator $Sim_{\mathcal{J}}$ is computationally indistinguishable from the judge's view in the real protocol execution. In the real protocol, the judge receives $t-1$ encrypted randomized Bloom filters derived from the clients' actual sets $\{S_i\}_{i\in[t-1]}$. In the simulation, the judge receives $t-1$ encrypted randomized Bloom filters derived from the simulated sets $\{\hat{S}_i\}_{i\in[t-1]}$. However, because the adversary does not have access to the server's secret decryption key, the IND-CPA security of the ElGamal encryption scheme guarantees that the adversary cannot distinguish between the filters generated from the real sets $\{S_i\}_{i\in[t-1]}$ and the filters generated from the simulated sets $\{\hat{S}_i\}_{i\in[t-1]}$.

Therefore, the simulated view is computationally indistinguishable from the real protocol view. The protocol is secure against a semi-honest judge. □

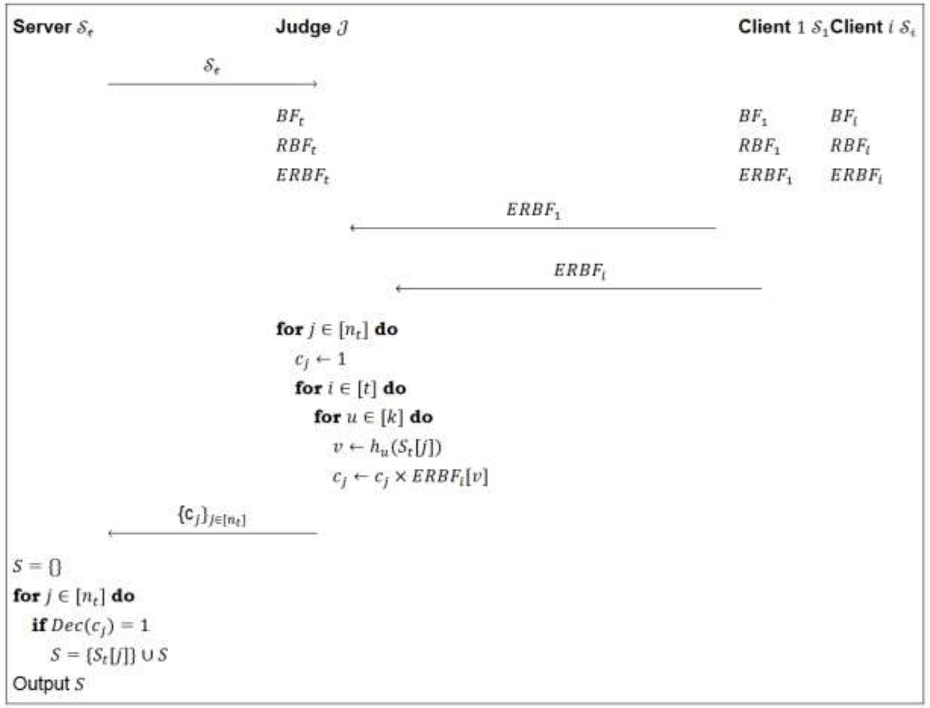


Figure 5.3: Authorized PSI with public parameters: prime number p, generator g, public key $pk = g^{sk}$ where sk is the server's secret key. BF_i represents the Bloom filter of party i, RBF_i represents their randomized Bloom filter and $ERBF_i$ their encrypted randomized Bloom filter.

Progress

APSI 2 – OPTIMIZED Active Judge

The Simulator (Sim_{S_t}) proceeds as follows:

1. Extract the set S_t^* sent by the malicious server to the judge.
2. Send S_t^* to the Ideal Functionality F_{APSI} and receive the true intersection $Z = S_t^* \cap \left(\bigcap_{i=1}^{t-1} S_i\right)$.
3. Honestly compute the Bloom Filter BF_t, Randomized Bloom Filter RBF_t and the Encrypted Randomized Bloom Filter $ERBF_t$, just as the real Judge would, based on S_t^*.
4. For each client $i \in [t-1]$:
   (a) Define a set $\hat{S}_i = Z$ to simulate S_i.
   (b) Compute the Bloom filter $B\hat{F}_i$ corresponding to $\hat{S}_i$ honestly by inserting each element into the m bins using the k hash functions.
   (c) Define the randomized Bloom filter $R\hat{B}F_i$ by sampling random group elements for each bin $v \in [m]$ where $B\hat{F}_i[v] = 0$, and copying over the 1s for bins where $B\hat{F}_i[v] = 1$.
   (d) Define the encrypted randomized Bloom filter $ER\hat{B}F_i$ by encrypting each bin $v \in [m]$ of $R\hat{B}F_i$ using the server's public key pk.
5. Aggregate the ciphertexts honestly as the real judge would: for each $j \in [n_t]$, compute c_j by homomorphically multiplying the corresponding hash bins from $ERBF_t$ and all $\{ER\hat{B}F_i\}_{i=1}^{t-1}$.
6. Send the aggregated ciphertexts $\{c_j\}_{j\in[n_t]}$ to the server.

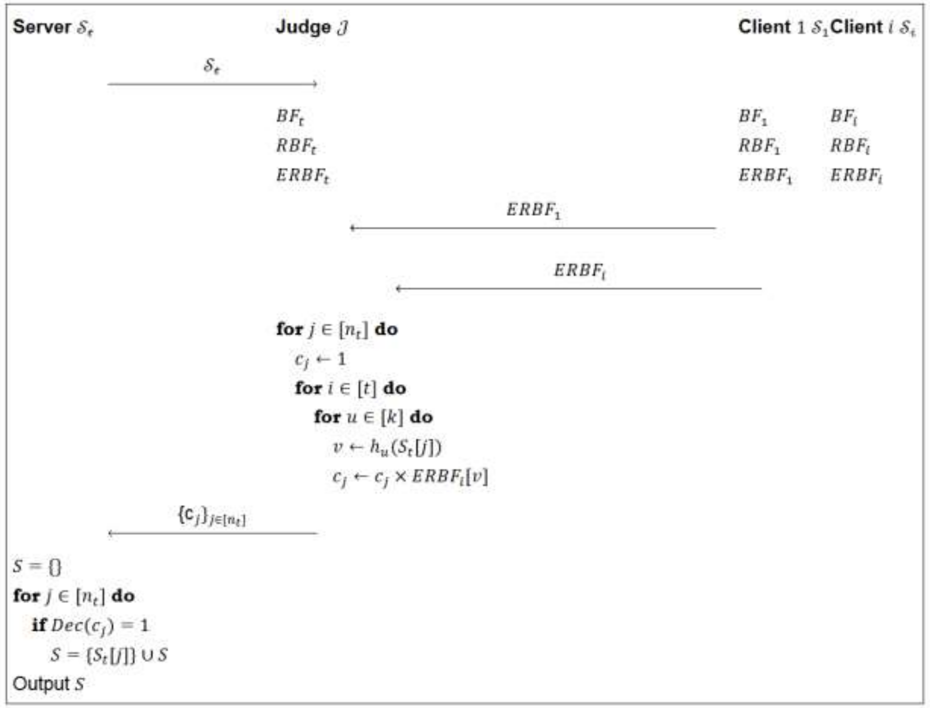
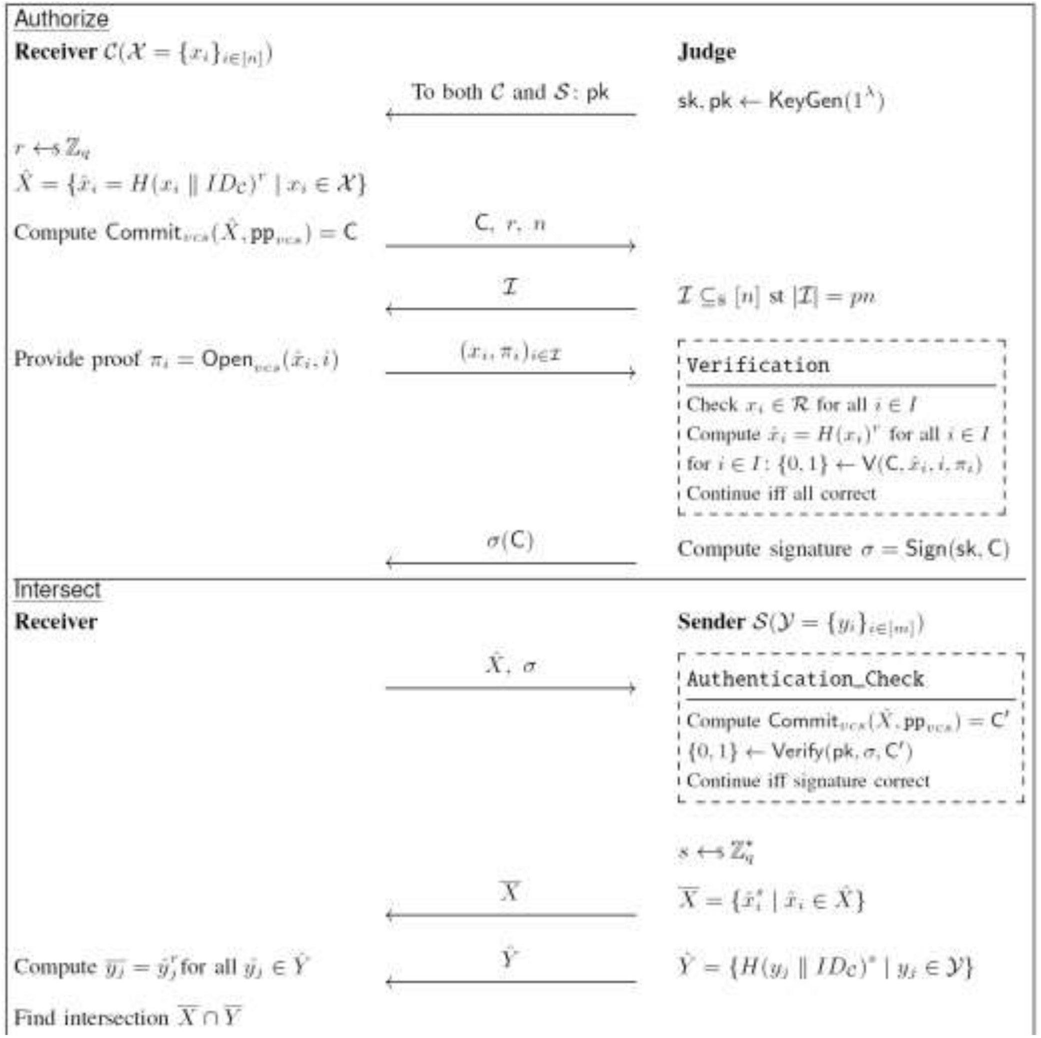


Figure 5.3: Authorized PSI with public parameters: prime number p, generator g, public key $pk = g^{sk}$ where sk is the server's secret key. BF_i represents the Bloom filter of party i, RBF_i represents their randomized Bloom filter and $ERBF_i$ their encrypted randomized Bloom filter.

PAPSI 2



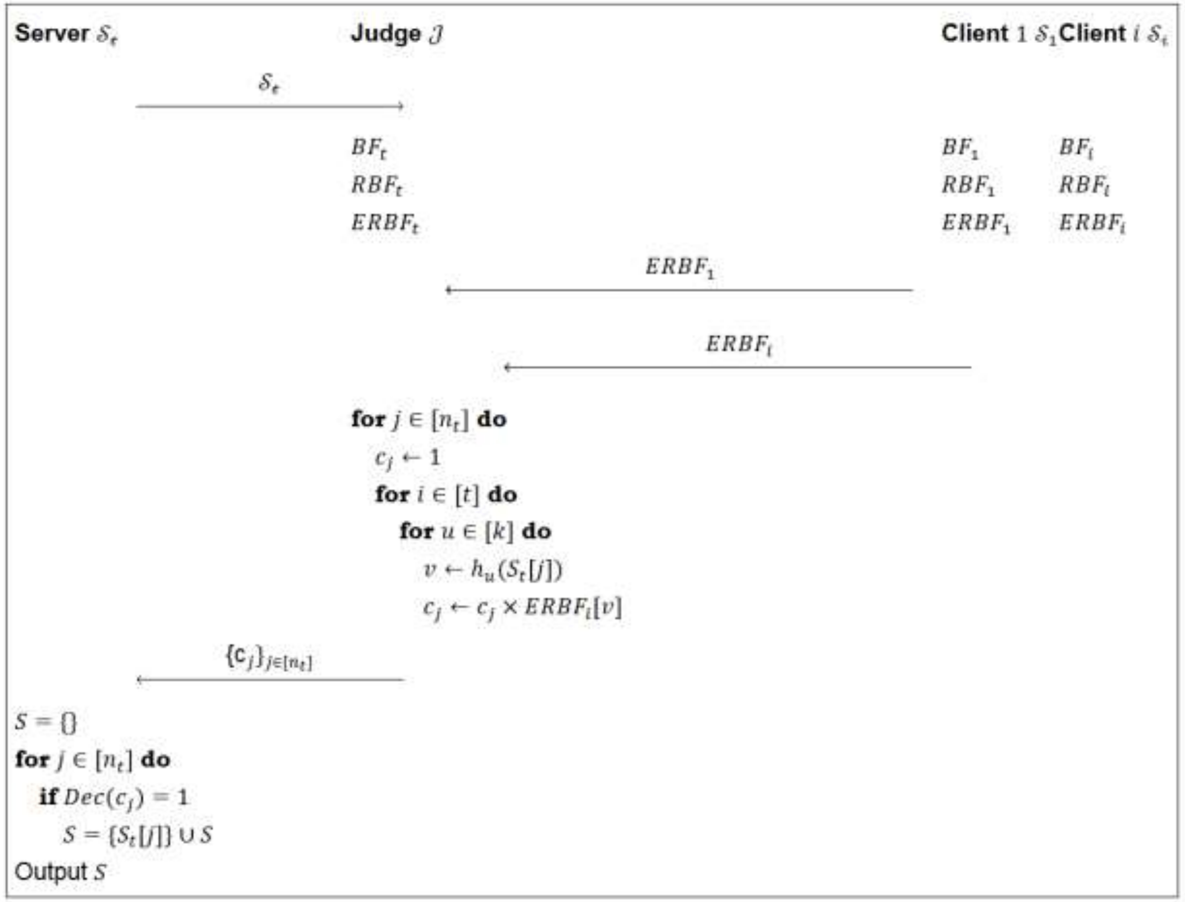


Figure 5.3: Authorized PSI with public parameters: prime number p, generator g, public key $pk = g^{sk}$ where sk is the server's secret key. BF_i represents the Bloom filter of party i, RBF_i represents their randomized Bloom filter and $ERBF_i$ their encrypted randomized Bloom filter.

Next Week

- Brainstorm on APSI1/PAPSI2
- Writing!

Thank you!

09-04-2026